

Módulo: Programación Multimedia y dispositivos móviles.

Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.

Actividad de Clase 3: Instalación de herramientas de desarrollo Multiplataforma.

1. Objetivo general.

Conocer, manejar y generar aplicaciones que multiplataforma con React Native, Flutter.

2. Metodología.

El alumnado realizará la app de forma individual.

3. Descripción.

React Native

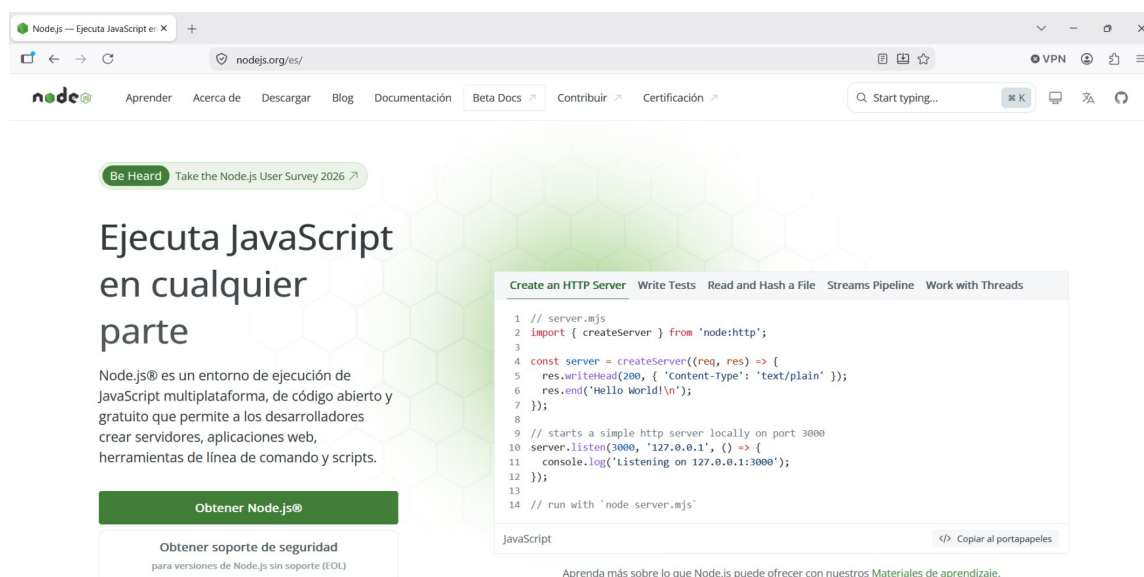


React Native es un Framework de desarrollo de aplicaciones móviles que permite crear aplicaciones multiplataforma utilizando JavaScript y React. Con React Native, puedes desarrollar aplicaciones para iOS y Android con una base de código común, lo que agiliza el proceso de desarrollo y permite reutilizar gran parte del código entre las plataformas.

Instalación

Realiza lo siguiente:

- 1) Antes de comenzar, debes tener instalado **Node.js** instalado en tu sistema. **React Native** también requiere que tengas instalado el administrador de paquetes **npm**, que generalmente se instala automáticamente con **Node.js**.
- 2) Para ellos, conéctate a la página de Node.js y descarga la versión para tu Sistema Operativo. :



- 3) Una vez descargado e instalado, NodeJs abre una terminal y ejecuta los siguientes comandos:

node -v

npm -v

```
C:\Windows\System32>node -v
v24.21.0
C:\Windows\System32>npm -v
11.16.0
C:\Windows\System32>
```

Instalación de React Native Expo.

- 4) Sino tienes una terminal (o cmd) abierto abre una e instala expo-cli ejecutando el siguiente comando.:

npm install -g expo-cli

- 5) Crea un directorio donde guardarás los proyectos realizados con React Native.
6) Accede a dicho directorio y vamos a crear nuestro primer proyecto con React Native.
7) Para ello ejecuta el siguiente comando:

npx create-expo-app holareact

- 8) En la siguiente imagen puedes ver un ejemplo de ejecución del comando:

```
C:\Users\jacin\react-native>npx create-expo-app holareact
Select an Expo SDK version: » Latest (SDK 57)
Creating holareact using the default template.

Tip:
  * npx create-expo-app --template to pick from other templates
  * npx create-expo-app --example to explore https://github.com/expo/examples

Downloaded and extracted project files.
> npm install
npm warn deprecated uuid@7.0.3: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid
11 (but be aware this version will likely be deprecated in 2028).

added 607 packages, and audited 608 packages in 58s

48 packages are looking for funding
  run `npm fund` for details

14 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

☑ Your project is ready!

To run your project, navigate to the directory and run one of the following npm commands.

- cd holareact
- npm run android
- npm run ios # you need to use macOS to build the iOS project - use the Expo app if you need to do iOS development without a Mac
- npm run web

C:\Users\jacin\react-native>
```

- 9) Una vez creado el proyecto entra en el directorio del proyecto.

cd holareact

```
C:\WINDOWS\system32\cmd. X + v
C:\Users\jacin\react-native>cd holareact
C:\Users\jacin\react-native\holareact>
```

- 10) Para ejecutar el servidor puedes ejecutar uno de estos dos comandos:

npm start

o

npm expo start

- 11) Esto iniciará el servidor de desarrollo y te proporcionará un código QR que puedes escanear con la aplicación Expo Go en tu dispositivo móvil para ver la aplicación en acción.

- 12) Apareciendo el siguiente menú con las opciones de ejecución.

```
C:\WINDOWS\system32\cmd. X + v
C:\Users\jacin\react-native\holareact>npm expo start
Starting project at C:\Users\jacin\react-native\holareact
React Compiler enabled
Starting Metro Bundler



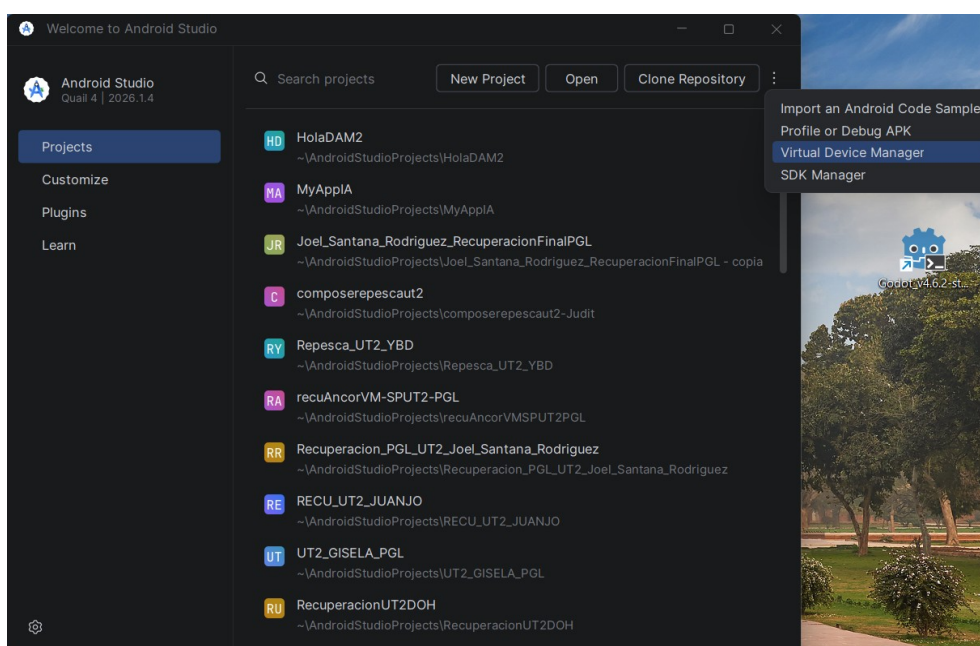
> Metro waiting on exp://192.168.1.53:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)
> Web is waiting on http://localhost:8081
> Using Expo Go
> Press s | switch to development build
> Press a | open Android
> Press w | open web
> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor
> Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.
```

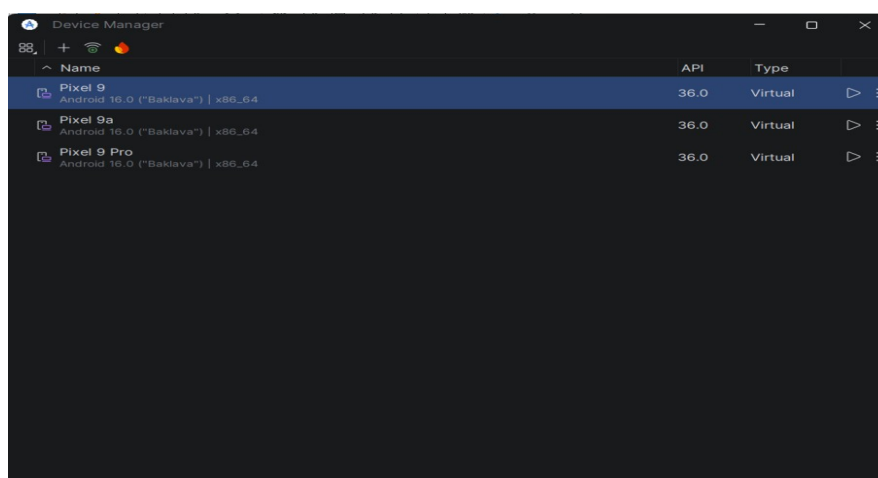
Ejecución en un Emulador Android con Android Studio.

13) Ya tienes instalado Android Studio. Y también debes tener configurado un un AVD (Android Virtual Device) dispositivo virtual. Para abrir el dispositivo virtual sin abrir Android Studio.

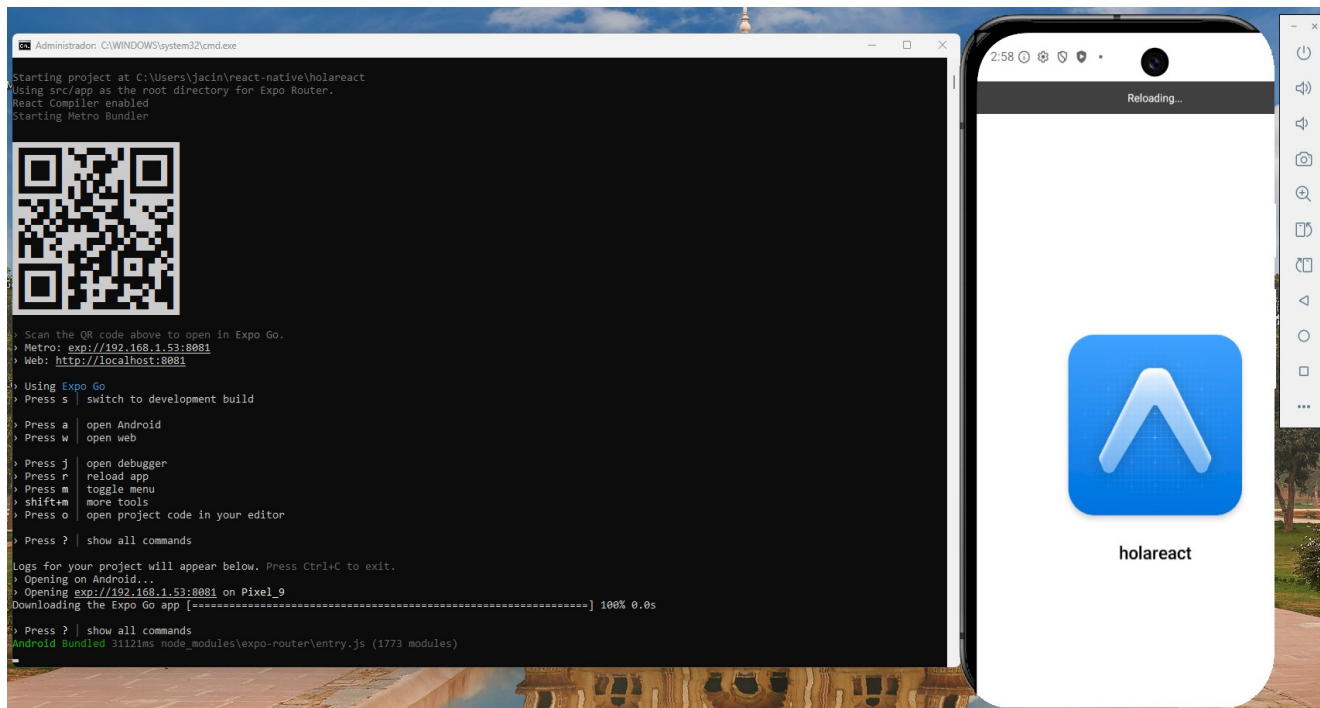
14) Para ello abre Android Studio, pincha sobre los tres puntos y luego selecciona Virtual Device Manager.



15) Ahora inicia el dispositivo donde quiere que se ejecuten las aplicación.

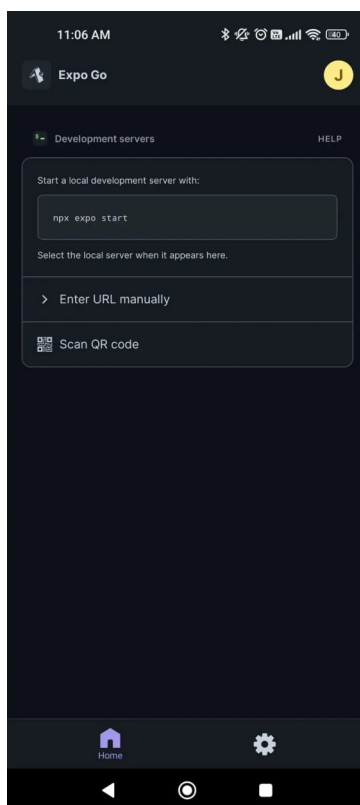


16) Ahora seleccionamos la opción **a** para que se ejecute en el emulador de **Android Studio**. Como puedes ver en la siguiente imagen.



Ejecución en un dispositivo Android.

- 17) Instalar **expo** en tu smartphone para pruebas (debe estar conectado a la misma red wifi que el equipo donde tienes instalado y configurado React Native Expo).
- 18) A continuación puedes ver una imagen de ejemplo:



Resetear Proyecto y modificarlo.

19) Para que construyas tu aplicación vas a quitar el código que tiene la aplicación y comenzar de nuevo con un nuevo proyecto. Ejecute el siguiente comando para restablecer el proyecto:

npm run reset-project

```
Administrador: Símbolo del sistema
C:\Users\jacin\react-native\holareact>npm run reset-project

> holareact@1.0.0 reset-project
> node ./scripts/reset-project.js

Do you want to move existing files to /example instead of deleting them? (Y/n): Y
  /example directory created.
  /src moved to /example/src.
  /scripts moved to /example/scripts.

  New /src/app directory created.
  src/app/index.tsx created.
  src/app/_layout.tsx created.

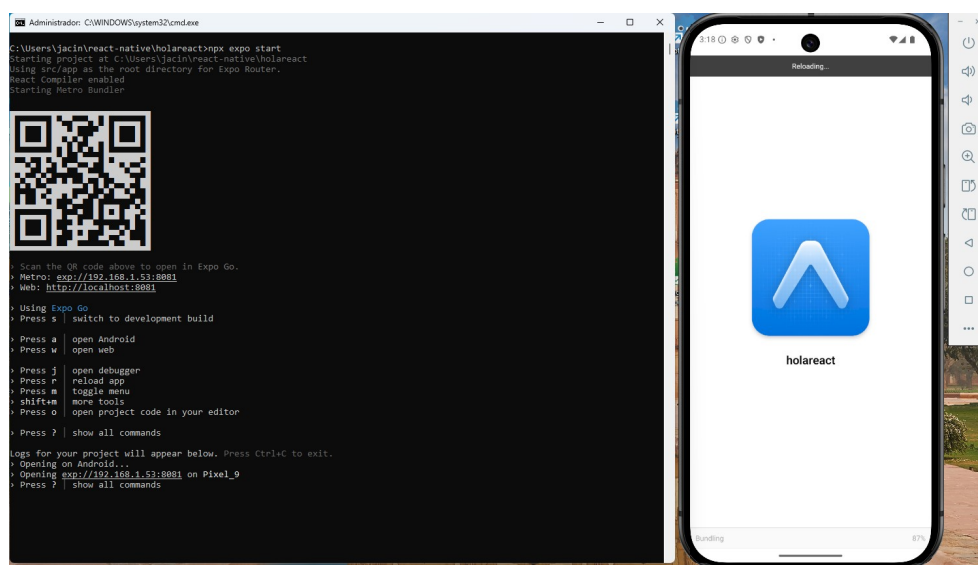
☑ Project reset complete. Next steps:
1. Run `npx expo start` to start a development server.
2. Edit src/app/index.tsx to edit the main screen.
3. Put all your application code in /src, only screens and layout files should be in /src/app.
4. Delete the /example directory when you're done referencing it.

C:\Users\jacin\react-native\holareact>
```

20) Inicia de nuevo el servidor ejecutando el comando:

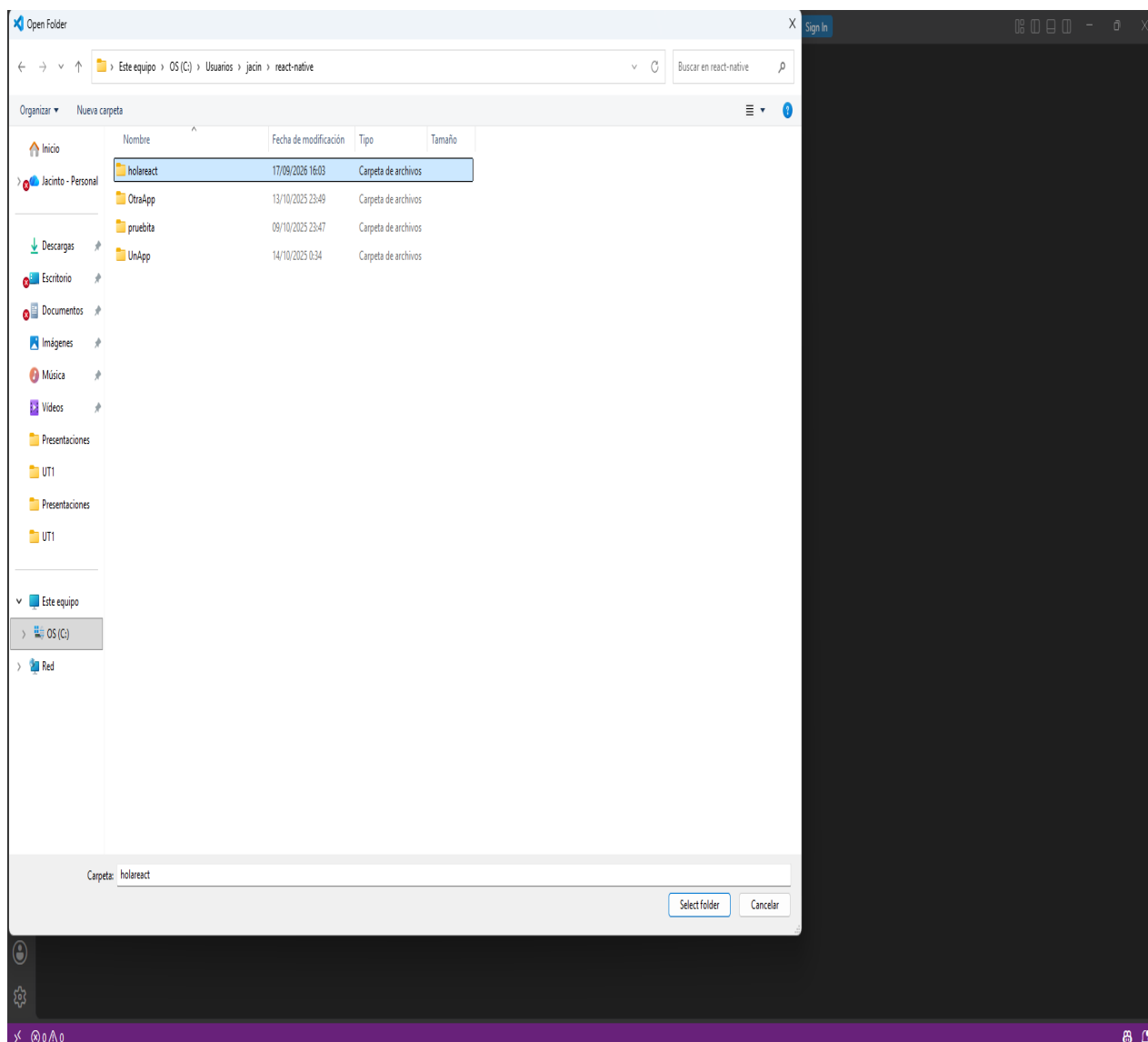
npx expo start

21) Y ejecuta la aplicación en Android (opción **a**)

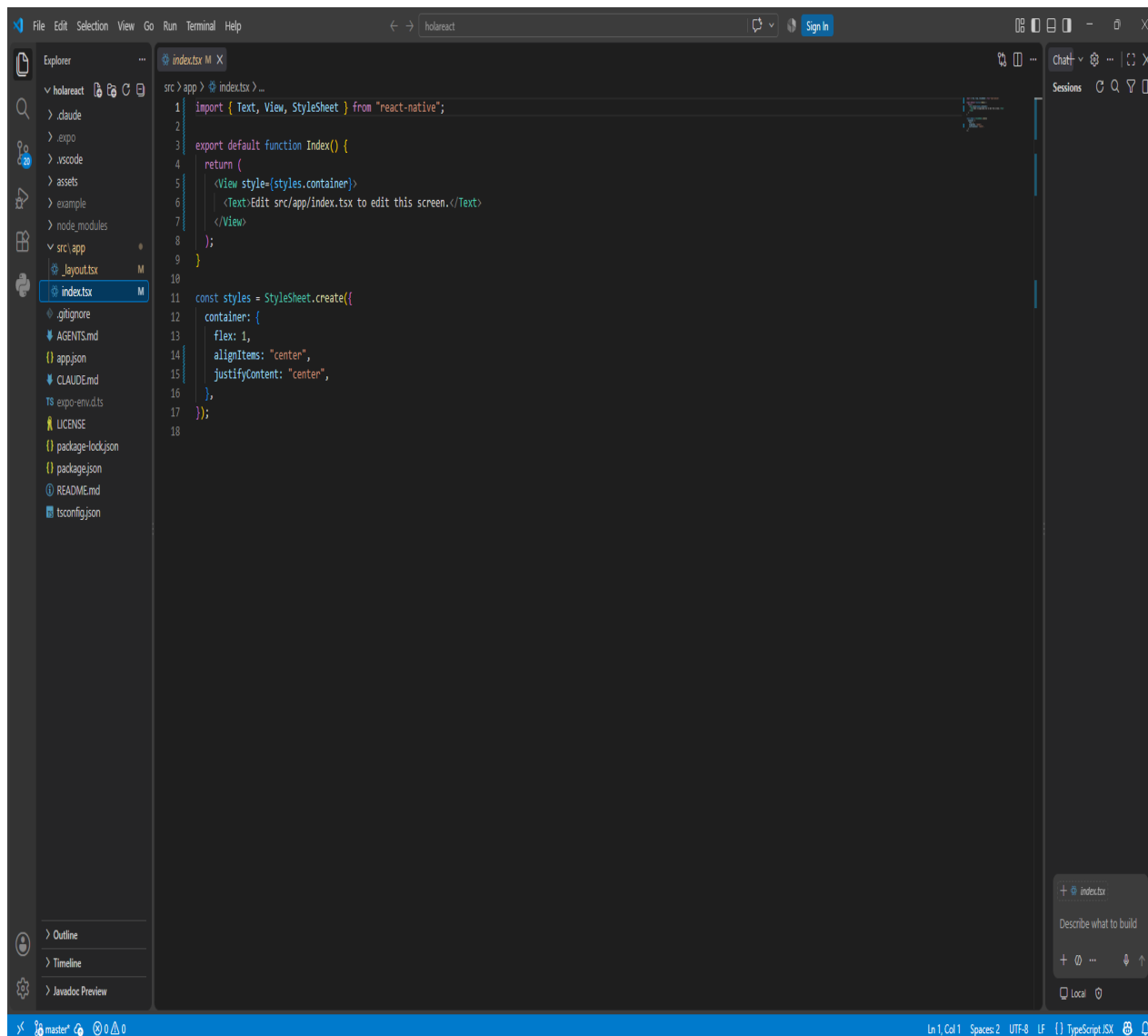


Editar el proyecto con Visual Studio Code.

22) Abre el Visual Studio Code y a continuación abre el directorio del proyecto que acabas de crear:



23) Accede a la carpeta **src/app** y luego edita el fichero **index.tsx**



```
1 import { Text, View, StyleSheet } from "react-native";
2
3 export default function Index() {
4   return (
5     <View style={styles.container}>
6       <Text>Edit src/app/index.tsx to edit this screen.</Text>
7     </View>
8   );
9 }
10
11 const styles = StyleSheet.create({
12   container: {
13     flex: 1,
14     alignItems: "center",
15     justifyContent: "center",
16   },
17 });
18
```

24) Modifica el texto por tu propio mensaje y añade mas texto.

Flutter.



Flutter es un SDK desarrollado por Google para crear aplicaciones móviles tanto para Android como para iOS (Apple). Fue desarrollado como un software para uso interno dentro de la compañía pero vieron el potencial que tenía y decidieron lanzarlo como proyecto de código libre.

Las principales ventajas que ofrece Flutter respecto a otras frameworks de desarrollo de aplicaciones multiplataforma:

- Compila en nativo, tanto en Android como en iOS.
- La creación interfaces gráficas es muy flexible, puedes combinar diferentes Widgets (elementos gráficos) para crear las vistas.
- El desarrollo es muy rápido, permite ver el resultado de forma instantánea mientras se escribe el código.

¿Cómo funciona Flutter?

Flutter utiliza un motor gráfico, llamado Skia, que renderiza en 2D los elementos gráficos. Flutter está escrito en C, C++ y, mayormente, en Dart. La capa del motor está escrito en C++ y la de los Widgets en Dart.

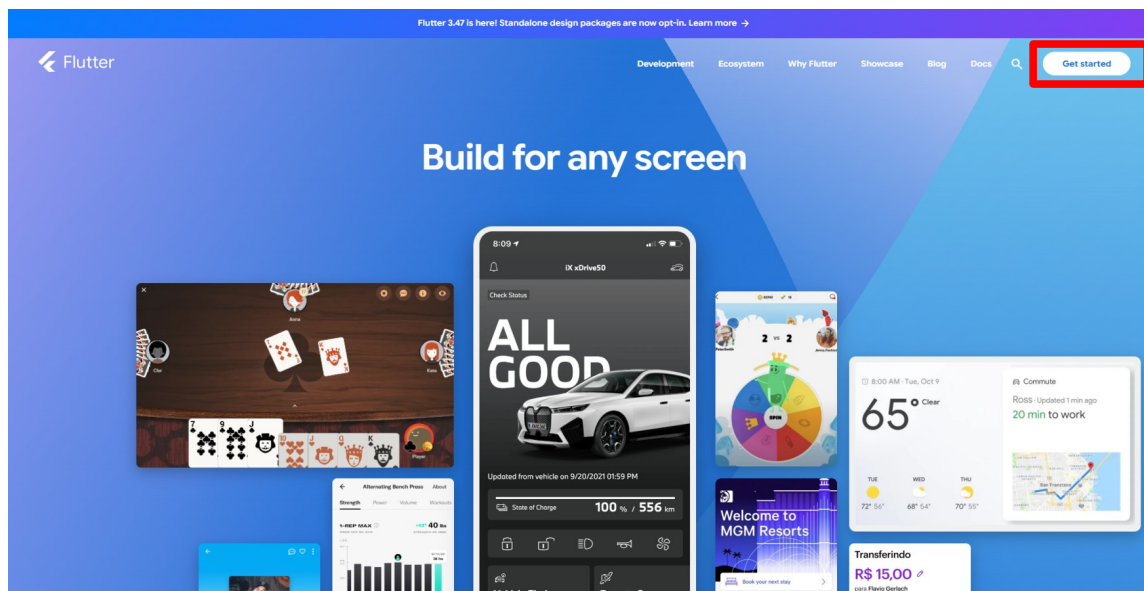
Widget es la palabra más usada cuando hacemos referencia a Flutter, se refiere a los elementos gráficos que componen una vista. Por ejemplo, un botón, un texto o una imagen. Aunque también hay Widgets más complejos que están formados por otros Widgets.

Flutter utiliza Dart como lenguaje de programación, también desarrollado por Google. Posiblemente, esta sea una desventaja, ya que Dart no es un lenguaje muy conocido ni usado por la comunidad de desarrolladores móviles (Swift, Objective-C, Kotlin, Java) y web (JavaScript, PHP, Java, Ruby...). Aunque es muy parecido a Java y C# (Microsoft), si se tiene experiencia con alguno de estos lenguajes es muy intuitivo empezar con Dart.

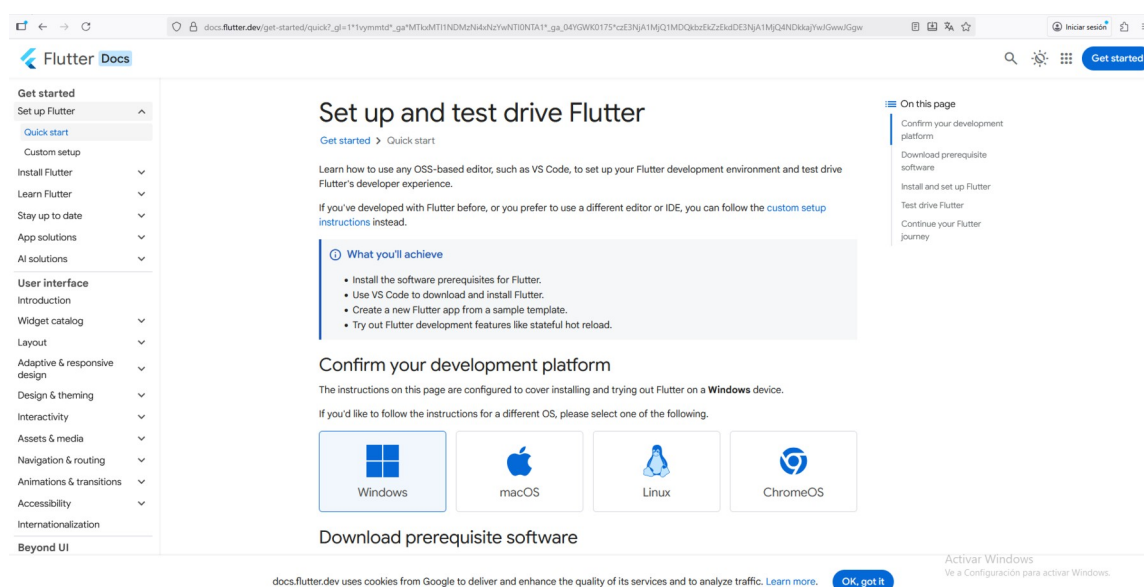
Google ha despedido trabajadores del Proyecto de Flutter, pero de momento la comunidad sigue detrás y todavía no podemos

Instalación de Flutter.

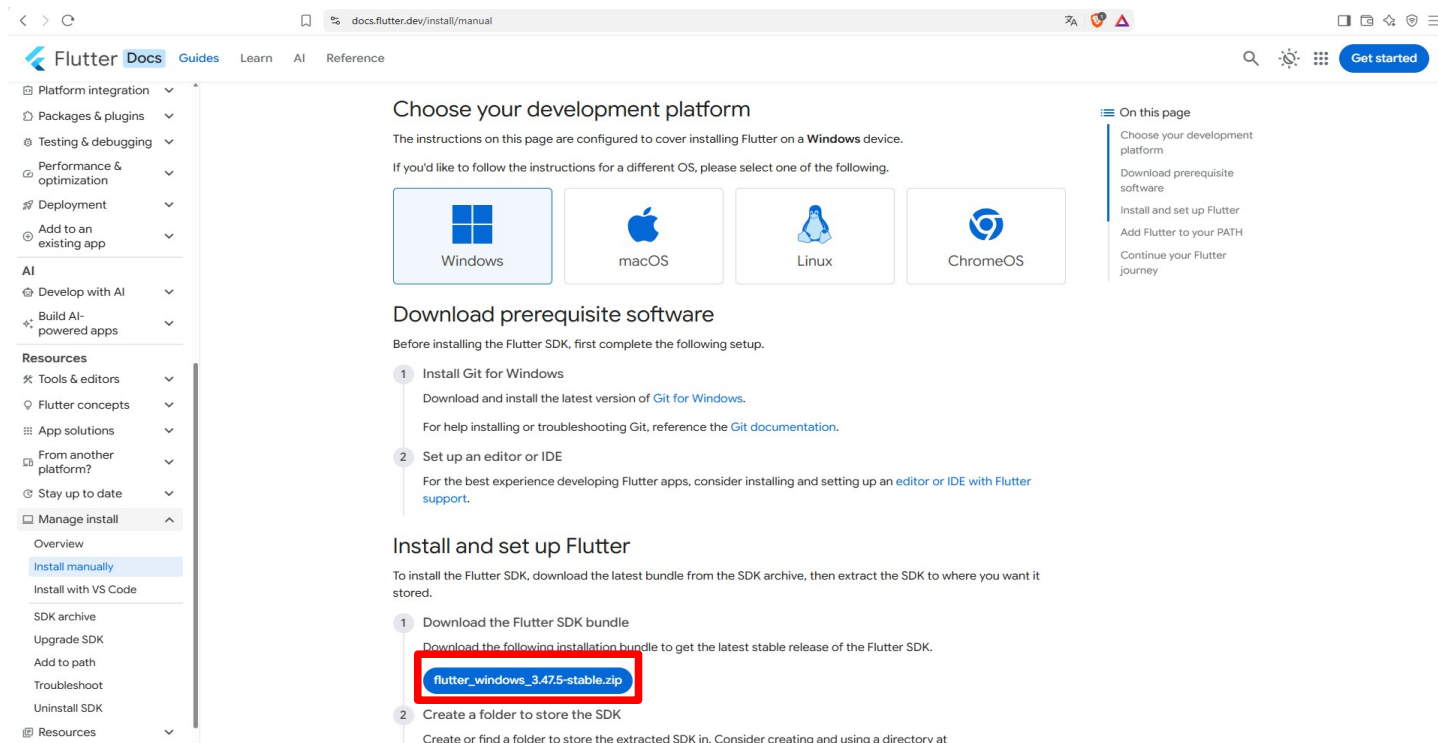
25) Visita la página de flutter. <https://flutter.dev/> y pulsa sobre Get Started



26) Descarga el último SDK de Flutter para el Sistema Operativo que vayas a utilizar, en el aula Windows pero si tienes un equipo con MacOS o Linux. Es preferible que hagas la descarga manual.



27) Es preferible que hagas la descarga manual. Y descargues el último SDK estable de Flutter.



28) Extrae el contenido del archivo zip en un directorio adecuado, como por ejemplo, [C:\flutter](#) o [C:\src\flutter](#). Este será tu directorio de instalación de Flutter. Evita ubicaciones como C:\Program Files que requieren elevados permisos de seguridad.

29) Editar las variables de entorno del sistema» y selecciona la opción que aparece. En la ventana del sistema, haz clic en «Variables de Entorno». Bajo «Variables del sistema», busca «Path» y selecciona «Editar». Haz clic en «Nuevo» y añade el path completo del directorio flutter\bin. Haz clic en «Aceptar» para cerrar las ventanas.

30) Ahora vamos a verificar de la Instalación para ello abre una nueva terminal o ventana de comandos (Ejecuta como administrador) . Escribe flutter doctor y presiona enter.

```
Administrador: Símbolo del sistema - flutter doctor
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.5, on Microsoft Windows [Versi n 10.0.26200.9457], locale es-ES)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[!] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
    X Android license status unknown.
      Run "flutter doctor --android-licenses" to accept the SDK licenses.
      See https://flutter.dev/to/windows-android-setup for more details.
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
[✓] Connected device (3 available)
[✓] Network resources

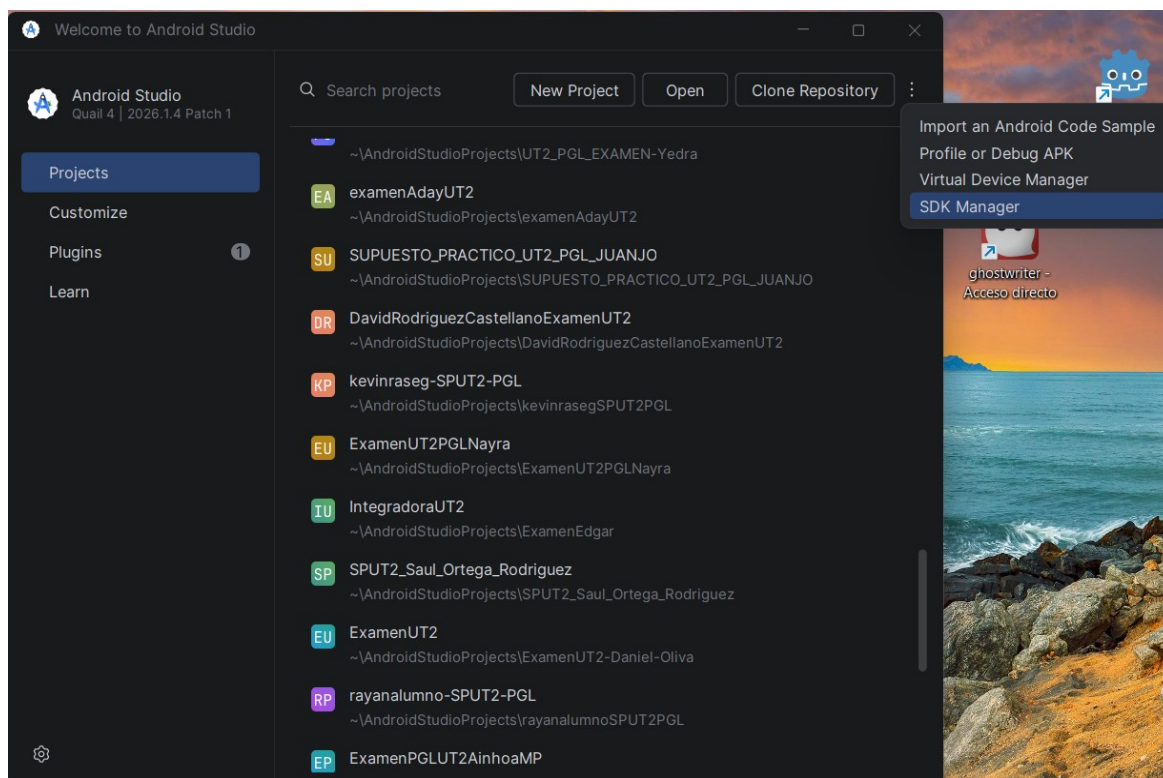
! Doctor found issues in 1 category.

C:\Windows\System32>
```

31) Flutter Doctor verificar  autom ticamente que tu entorno est  listo para construir apps con Flutter. Sigue las indicaciones para instalar herramientas adicionales si es necesario. En nuestro caso tenemos Android Studio instalado y vamos a instalar lo que nos sugiere Flutter Doctor.

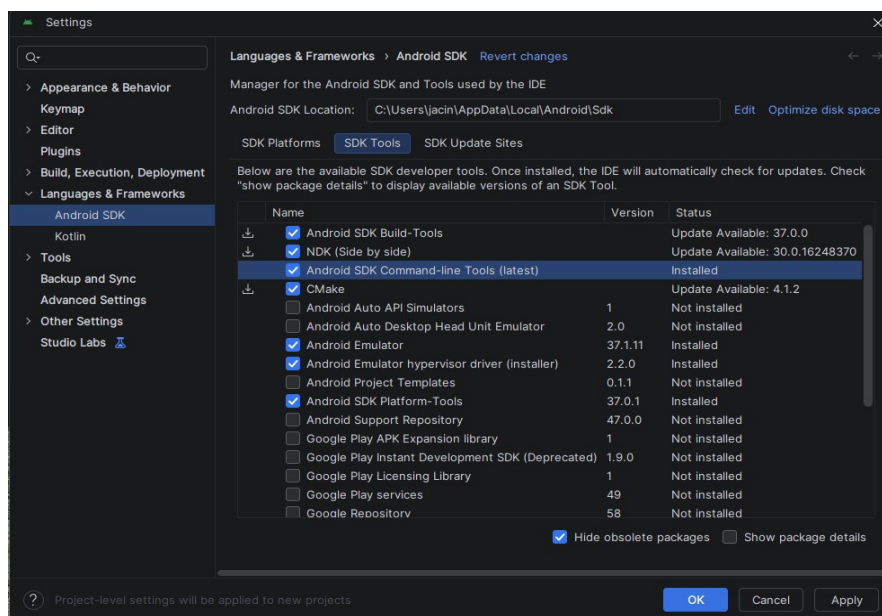
32) En este caso, vamos a a adir el cmdline-tools en Android Studio.

33) Abre Android Studio y haz clic en "More Actions" (M s acciones) en la pantalla de bienvenida, luego selecciona "SDK Manager".



34) En la ventana de Preferencias, ve a la pestaña "SDK Tools".

35) Marca las casillas siguientes (sino están seleccionadas): **Android SDK Command-line Tools (latest)**", **NDK (Side BY side)**, **Cmake**, **Android SDK Platform-Tools** y **Android SDK Build-Tools**. Haz clic en "Apply" o "OK" para iniciar la instalación. Reinicia Android Studio para completar el proceso.



36) Ahora para añadir la licencia ejecutamos el siguiente comando **flutter doctor android-licenses** para aceptar las licencias del SDK. Acepta las licencias. Y una vez finalizado vuelve a ejecutar el **flutter doctor** y comprueba que todo este bien.

```
Administrador: Símbolo del sistema - flutter doctor - flutter doctor-adroid-licenses - flutter doctor adroid-licenses - flutter doctor-...
C:\Windows\System32>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.5, on Microsoft Windows [Versión 10.0.26200.9457], locale es-ES)
[✓] Windows Version (Windows 11 or higher, 25H2, 2009)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Chrome - develop for the web
[✓] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!

C:\Windows\System32>
```

37)

Ahora crea un directorio para los proyectos de flutter dentro del directorio de tu usuario. Dentro de este directorio ejecuta el siguiente comando:

flutter create nombre_de_tu_app

38) Una vez creado, ve al directorio de tu app con **cd nombre_de_tu_app**.

39) Recuerda iniciar el emulador con el dispositivo virtual antes.

- ```

C:\Users\jacin\Flutter>flutter create hola_flutter
Creating project hola_flutter...
Resolving dependencies in 'hola_flutter'...
Downloading packages...
Got dependencies in 'hola_flutter'.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/Flutterdev

In order to run your application, type:

$ cd hola_flutter
$ flutter run

Your application code is in hola_flutter\lib\main.dart.

C:\Users\jacin\Flutter>cd hola_flutter

C:\Users\jacin\Flutter>flutter run
Launching lib\main.dart on sdk gphone64 x86_64 in debug mode...
WARNING: A restricted method in JavaLangSystem has been called
WARNING: java.lang.System::load has been called by net.rubygrapefruit.platform.internal.NativeLibraryLoader in an unnamed module (file:/C:/Users/jacin/.gradle/wrapper/dists/gradle-9.3.1-at190tr96686zfvq4mndrblj8/gradle-9.3.1/lib/native-platform-9.22-milestone-29.jar)
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: The SDK Manager CLI tool (sdkmanager) is deprecated. Android CLI will be used instead.
The 'android' binary can also be found in the cmdline-tools directory, and 'android sdk' is the replacement for 'sdkmanager'.
To learn more about the android sdk and how to use it, see the documentation (https://d.android.com/tools/agents/android-cli)

Package ndk not found.
Package 28.2.1367358 not found.

FAILURE: Build failed with an exception.

* Where:
Build file 'C:\Users\jacin\Flutter\hola_flutter\android\build.gradle.kts' line: 19

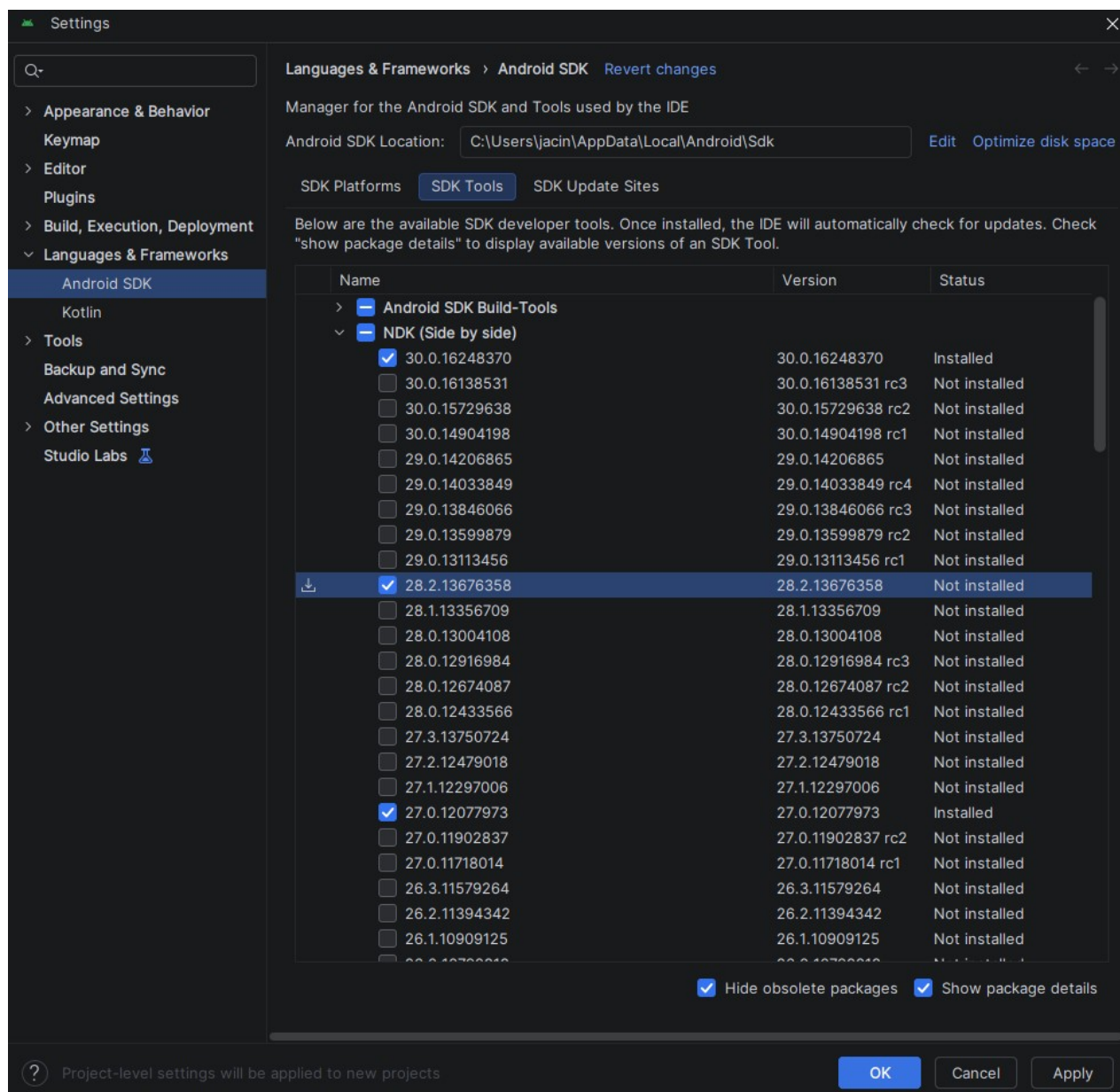
* What went wrong:
A problem occurred configuring project ':app'.
Process 'command "C:\Users\jacin\AppData\Local\Android\sdk\cmdline-tools\latest\bin\sdkmanager.bat"' finished with non-zero exit value -1873748791 (NTSTATUS 0xC0000409)

* Try:
Run with --stacktrace option to get the stack trace.
Run with --info or --debug option to get more log output.
Run with --scan to get full insights from a Build Scan (powered by Develocity).

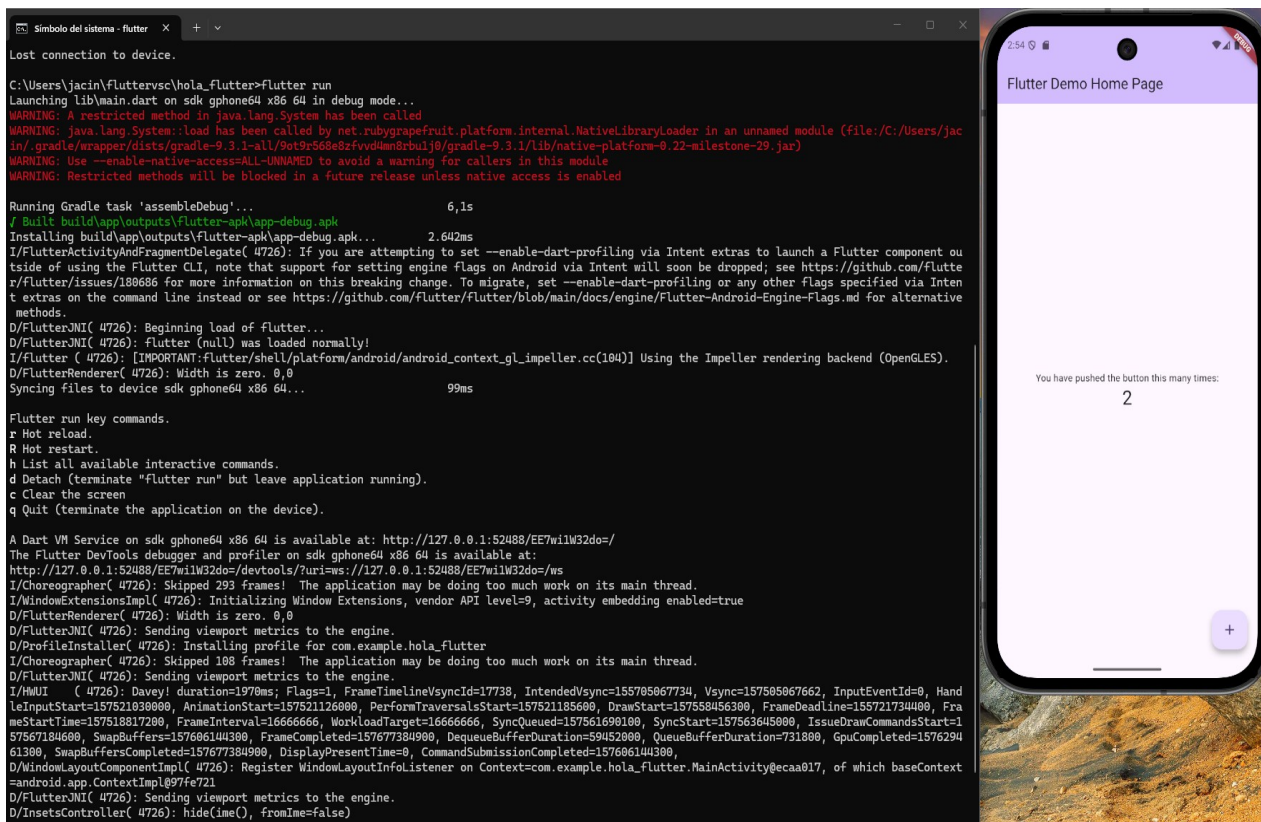
```

- 42) Vamos a añadir el paquete ndk con la versión que nos solicita. Para ello vuelve a abrir Android Studio y haz clic en "More Actions" (Más acciones) en la pantalla de bienvenida, luego selecciona "SDK Manager". En la ventana de Preferencias, ve a la pestaña "SDK Tools". Marca la casilla **NDK (Side BY side)** y selecciona **Show package details** y selecciona la versión **28.2.13676358**. Haz clic en "Apply" o "OK" para iniciar la instalación. Reinicia Android Studio para completar el proceso.



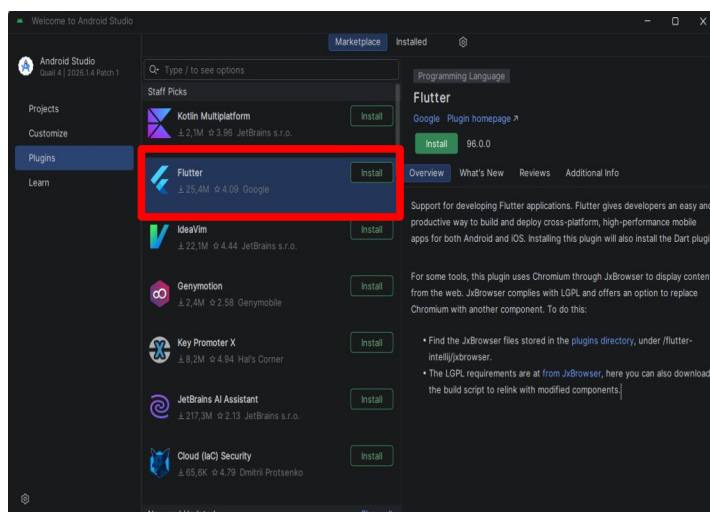


- 43) Ve de nuevo al directorio de tu app con `cd nombre_de_tu_app`.
- 44) Recuerda iniciar el emulador con el dispositivo virtual antes.
- 45) Ejecuta **flutter run** para iniciar tu app en el emulador o dispositivo conectado.
- 46) Ahora puedes ver como se inicia la app.

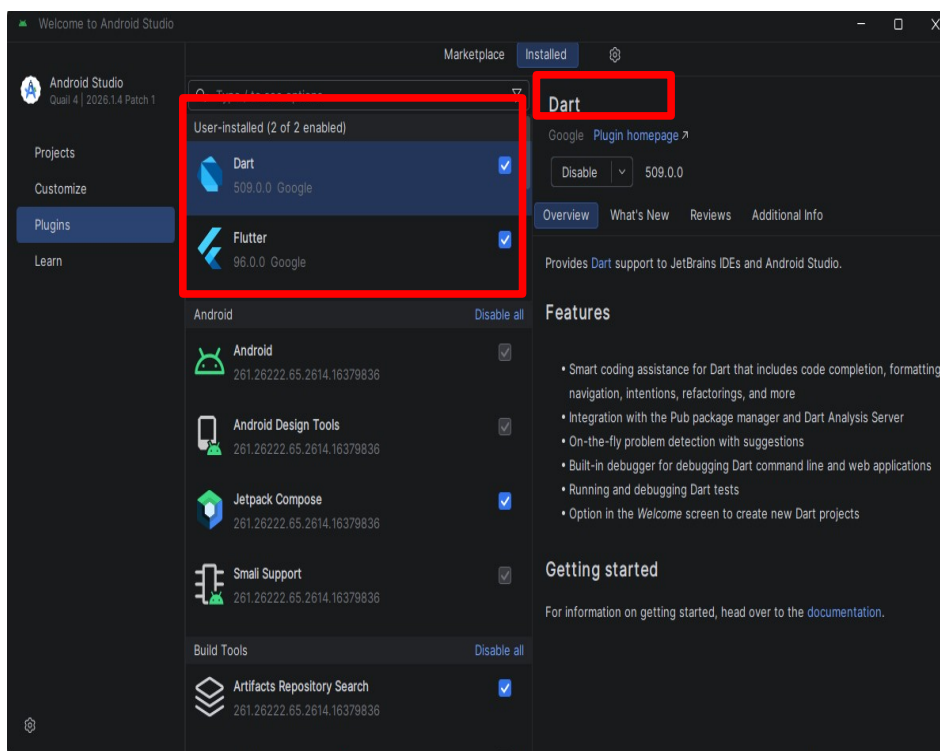


## Instalación de Flutter en Android Studio.

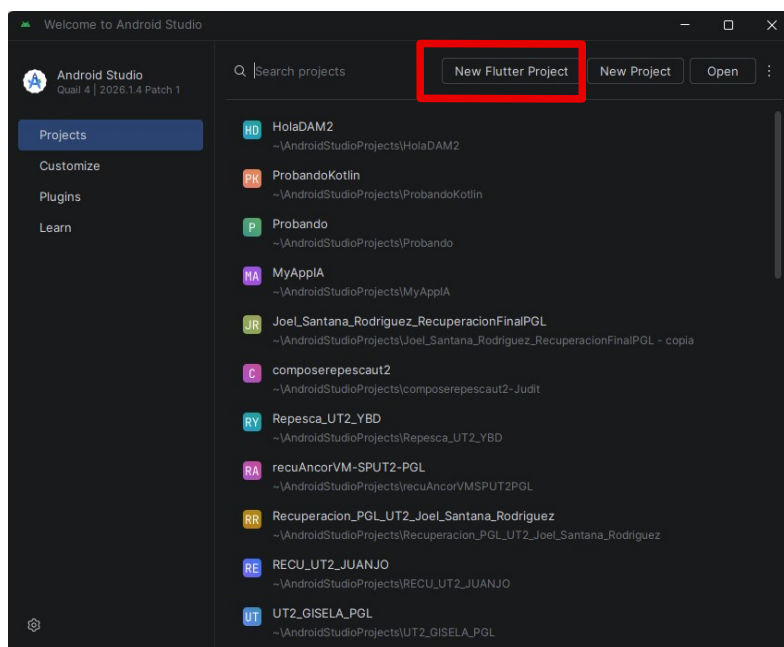
47) Para incorporar Flutter en Android Studio vamos a instalar dos plugin, el de **Flutter** y el de **Dart**. Para ello ejecuta Android Studio sin crear Proyecto. Y pulsa sobre **Plugins**. Y nos aparece Marketplace y buscamos Flutter y damos a Install



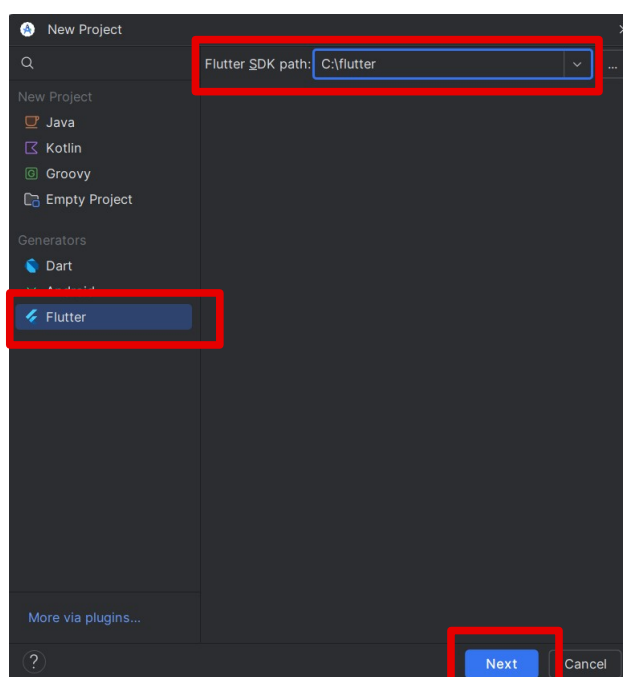
48) Luego damos a Restart IDE y pulsamos sobre Installed y veremos que se ha instalado los plugins de Flutter y Dart.



49) Una vez instalados los plugin de Flutter y Dart, pulsamos sobre Projects y nos aparecerá una nueva opción (botón) que pone **New Flutter Project**.



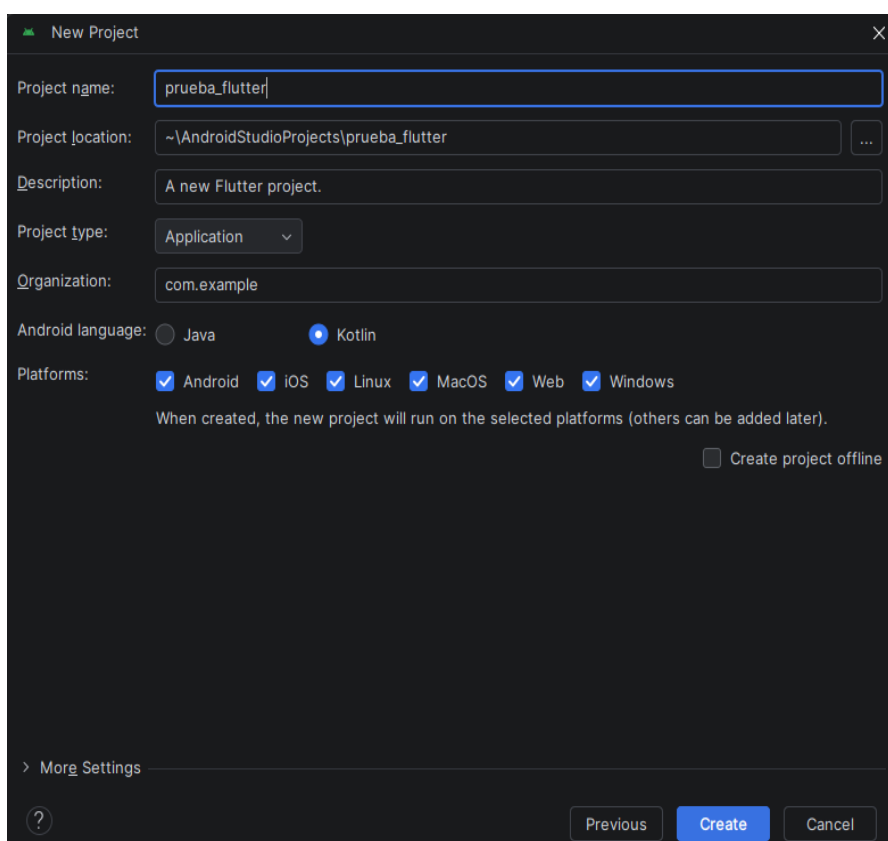
50) Pulsamos sobre dicho botón para crear un Proyecto de Flutter. En la siguiente ventana seleccionamos en la opción de New Project Flutter y en en Flutter path buscamos la carpeta donde descomprimos el SDK de Flutter que descargamos anteriormente. ([C:\flutter](#) en mi caso adapta la ruta a tu caso).



51) A continuación pulsa sobre el botón Next. El nombre del Proyecto debe cumplir la normas de paquetes en Dart. que deben ser minúsculas y usar guiones bajos para separar palabras, como en `mi_paquete_1`. También deben consistir únicamente en letras latinas básicas y dígitos ([a-z0-9\_]). Además, el nombre no debe ser una palabra reservada de Dart y debe seguir las convenciones de un identificador de Dart válido (no puede empezar con un dígito).

52) Mas información en la siguiente página, donde se especifica como deben ser los nombres de los paquetes en Dart.

[https://dart.dev/tools/linter-rules/package\\_names](https://dart.dev/tools/linter-rules/package_names)



New Project

Project name:

Project location:  ...

Description:

Project type:

Organization:

Android language: ☐ Java ☒ Kotlin

Platforms: ☒ Android ☒ iOS ☒ Linux ☒ MacOS ☒ Web ☒ Windows

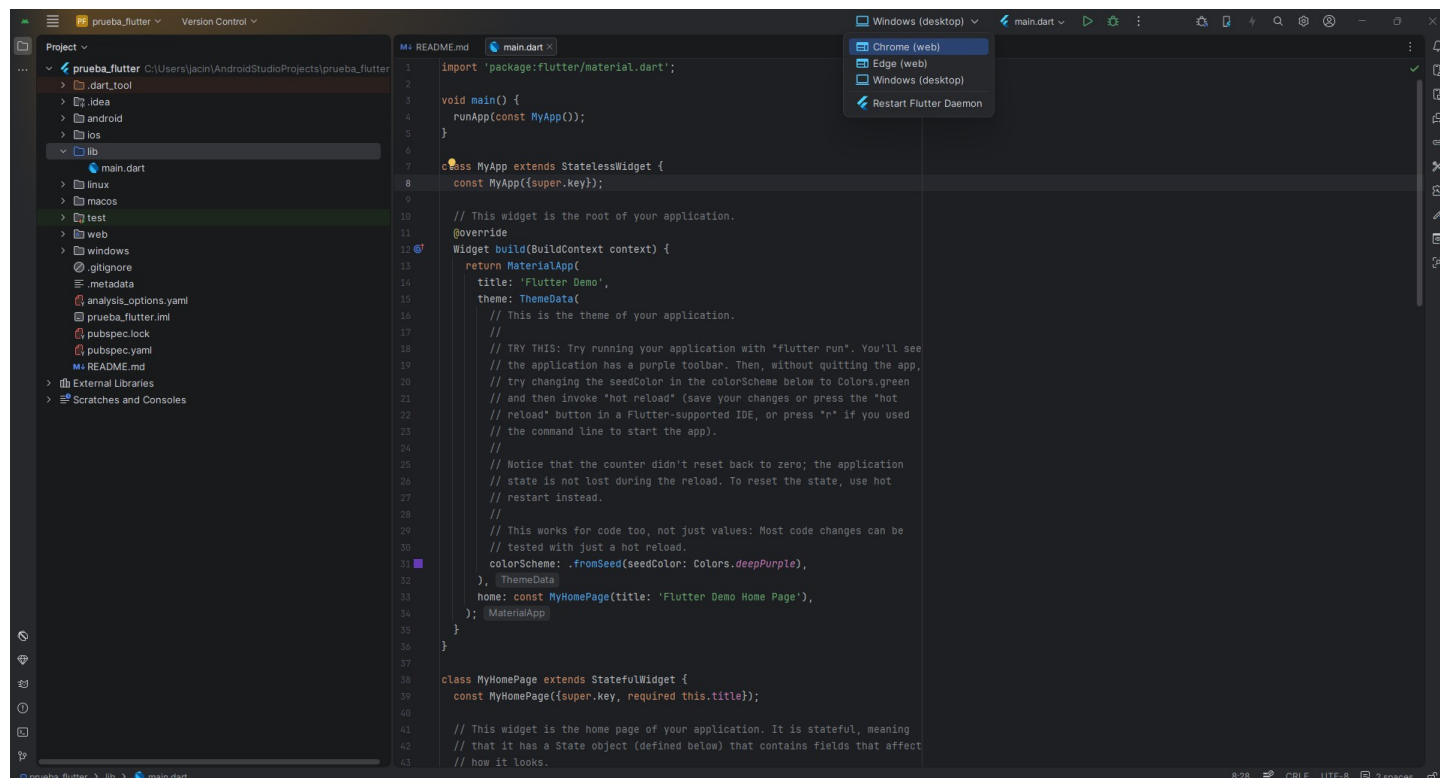
When created, the new project will run on the selected platforms (others can be added later).

☐ Create project offline

> More Settings

53) Y le damos a botón Create y nos creará el Proyecto. Tardará en el proceso de creación. Una vez creado nos aparecerá una ventana similar a la siguiente:





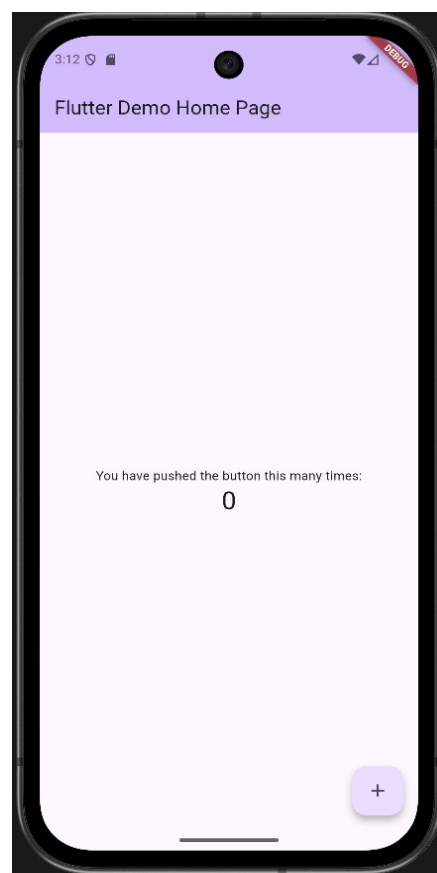
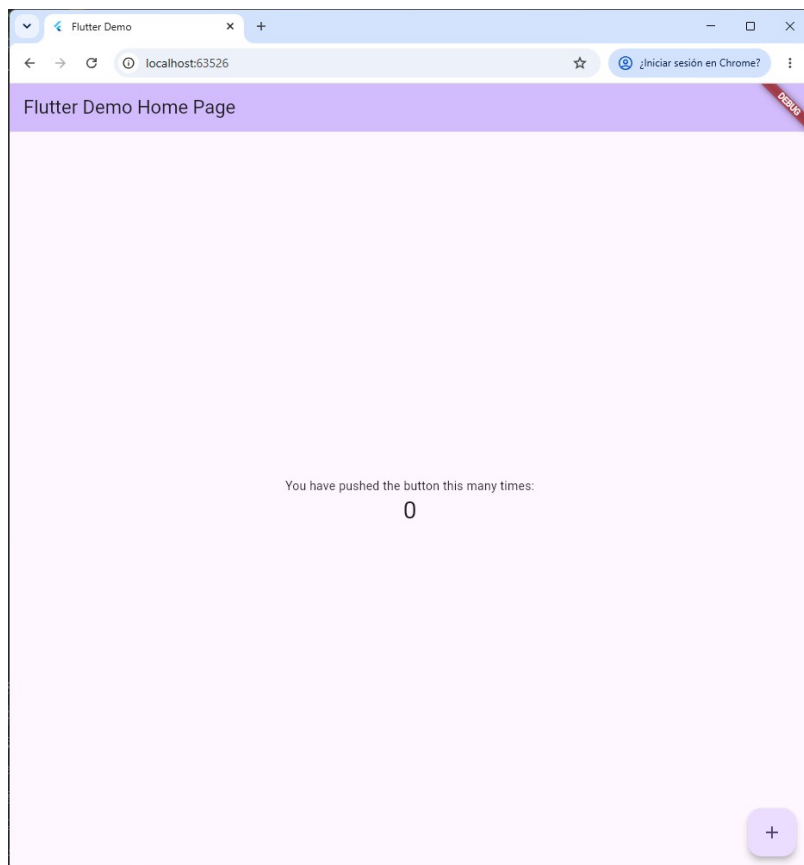
```

1 import 'package:flutter/material.dart';
2
3 void main() {
4 runApp(const MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8 const MyApp({super.key});
9
10 // This widget is the root of your application.
11 @override
12 Widget build(BuildContext context) {
13 return MaterialApp(
14 title: 'Flutter Demo',
15 theme: ThemeData(
16 // This is the theme of your application.
17 // TRY THIS: Try running your application with "flutter run". You'll see
18 // the application has a purple toolbar. Then, without quitting the app,
19 // try changing the seedColor in the colorScheme below to Colors.green
20 // and then invoke "hot reload" (save your changes or press the "hot
21 // reload" button in a Flutter-supported IDE, or press "r" if you used
22 // the command line to start the app).
23 // Notice that the counter didn't reset back to zero; the application
24 // state is not lost during the reload. To reset the state, use hot
25 // restart instead.
26 // This works for code too, not just values: Most code changes can be
27 // tested with just a hot reload.
28 colorScheme: .fromSeed(seedColor: Colors.deepPurple),
29),
30 home: const MyHomePage(title: 'Flutter Demo Home Page'),
31);
32 }
33
34 class MyHomePage extends StatefulWidget {
35 const MyHomePage({super.key, required this.title});
36
37 // This widget is the home page of your application. It is stateful, meaning
38 // that it has a State object (defined below) that contains fields that affect
39 // how it looks.

```

54) En la carpeta lib está código Dart (main.dart) y como vemos sin cambiar dicho código lo podemos ejecutar en Web (Chrome y Edge, y como tenemos activo el emulador del Pixel 9 también en este dispositivo). Como en el equipo que estoy utilizando he instalado Visual Studio y las librerías de Visual C++ también me permite ejecutarlo en Windows.

55) A continuación puedes ver imágenes de la ejecución de mismo código (una aplicación contador donde si pulsamos el más se incrementa y muestra el valor, en un navegador web y un emulador de Android, concretamente un Pixel 9.

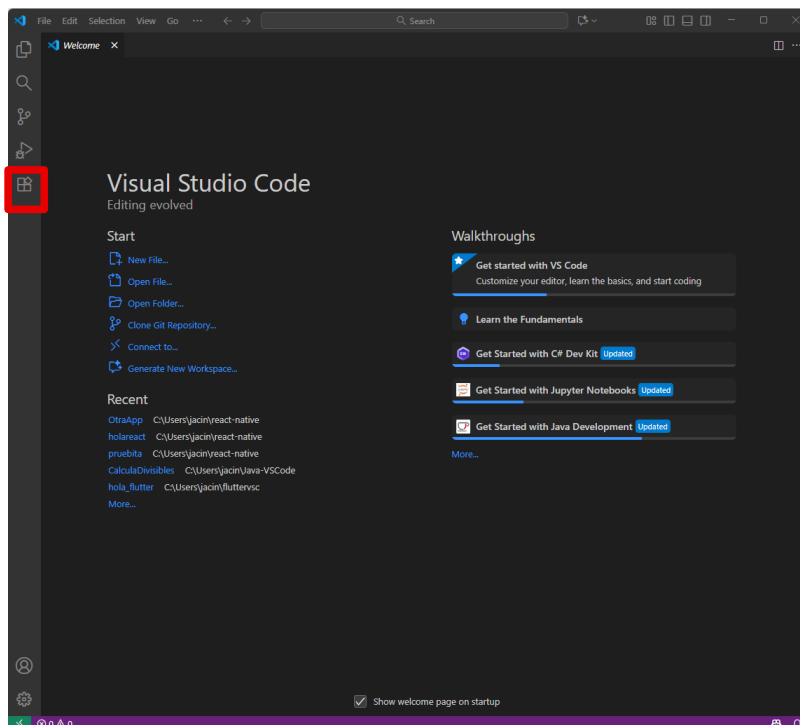


56) Modifica el texto por tu propio mensaje y añade mas texto.

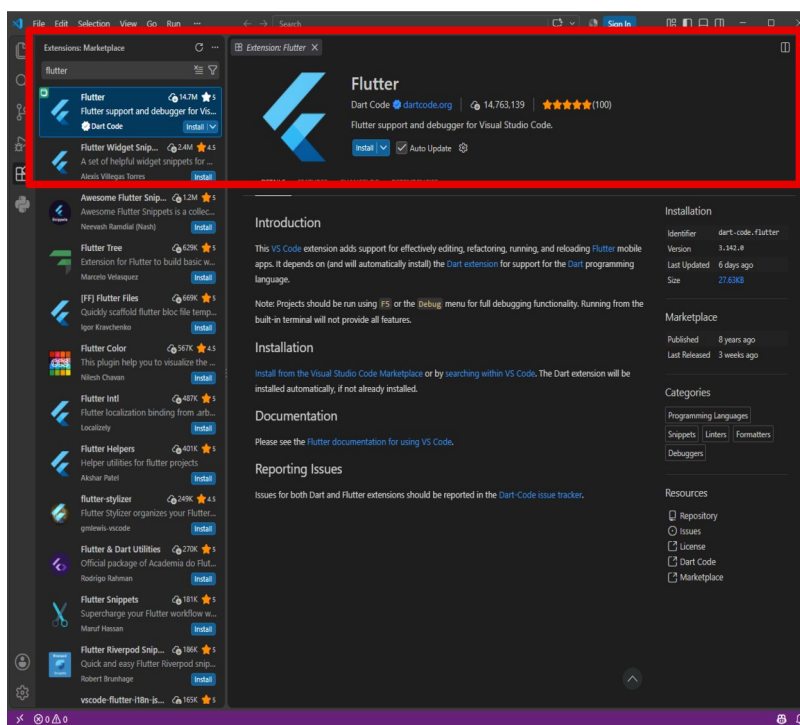


## Instalación de Flutter en Visual Studio Code.

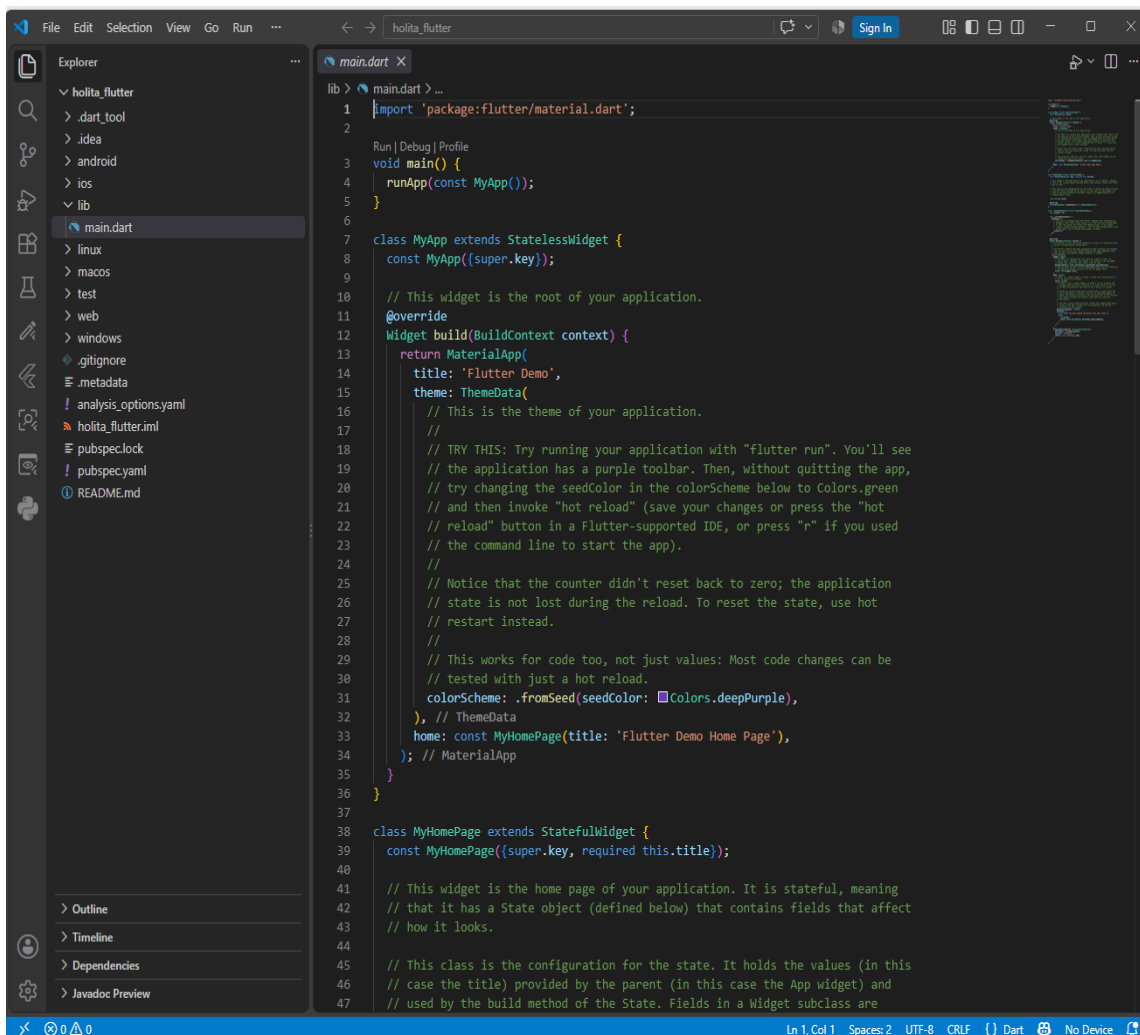
57) Abrimos el editor Visual Studio Code y pulsamos sobre las extensiones.



58) Ahora buscamos Flutter, la seleccionamos y el instamos dicha extensión.



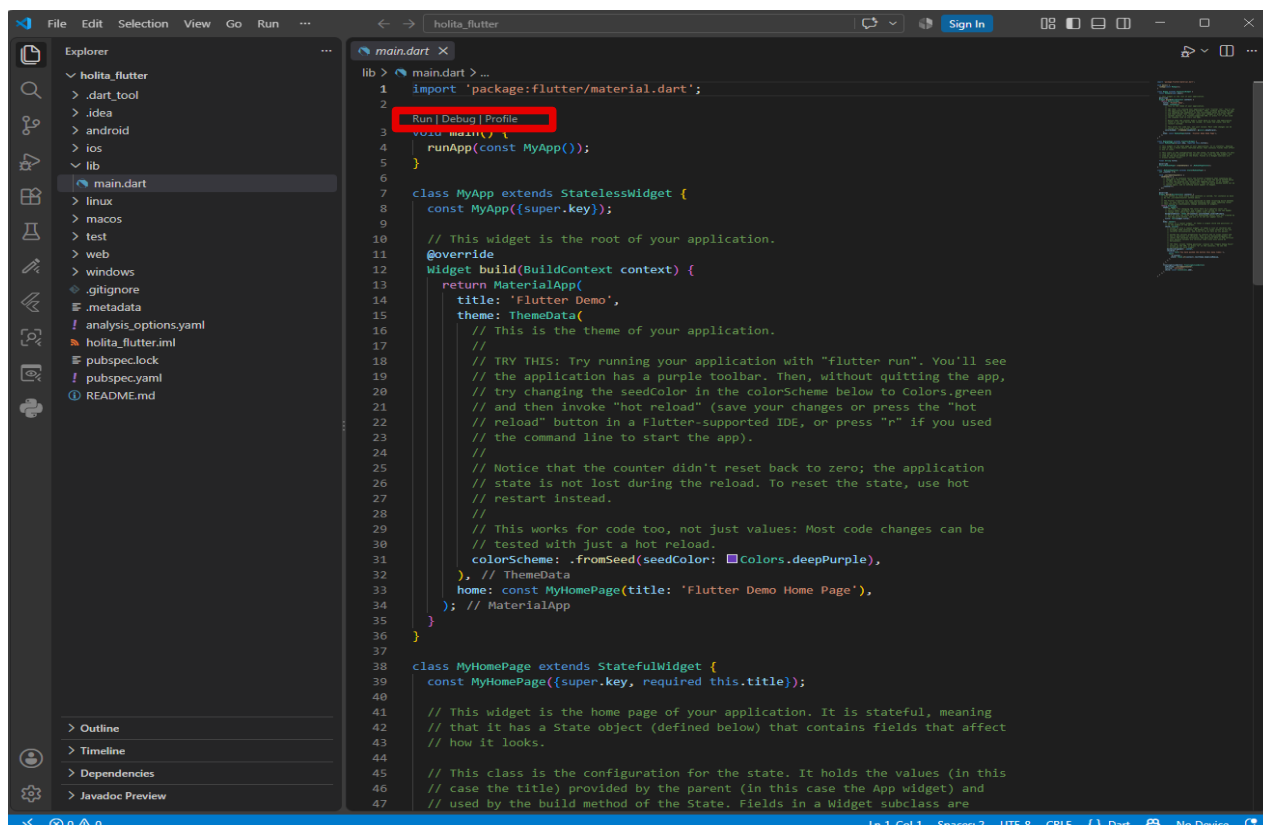
- 59) Ahora buscamos la extensión Dart y comprobamos que está instalado.
- 60) Ahora reinicia Visual Studio Code para que los cambios surtan efecto.
- 61) Ahora vamos a crear un nuevo Proyecto de Flutter. Para ello Abre la paleta de comandos con **F1** o ejecuta **Ctrl+Shift+P** (Windows) o **Shift+Cmd+P** (Mac).
- 62) Escribe y selecciona **Flutter: New Project**, a continuación selecciona **Application** como tipo de proyecto.
- 63) Selecciona una carpeta donde se guardará el proyecto. Y dale un nombre a tu proyecto (en minúsculas y sin espacios). Recuerda la nomenclatura de los paquetes en Dart.
- 64) Una vez creado el proyecto podemos ver la estructura de directorios de nuestro Proyecto en Flutter. Verás que la estructura es igual a que nos aparecía en Android Studio.



The screenshot shows the Visual Studio Code interface with a Flutter project named 'holita\_flutter'. The Explorer panel on the left shows the project structure, including directories like .dart\_tool, .idea, android, ios, lib, linux, macos, test, web, windows, .gitignore, .metadata, analysis\_options.yaml, holita\_flutter.iml, pubspec.lock, pubspec.yaml, and README.md. The main editor displays the main.dart file with the following code:

```
lib > main.dart > ...
1 import 'package:flutter/material.dart';
2
3 void main() {
4 runApp(const MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8 const MyApp({super.key});
9
10 // This widget is the root of your application.
11 @override
12 Widget build(BuildContext context) {
13 return MaterialApp(
14 title: 'Flutter Demo',
15 theme: ThemeData(
16 // This is the theme of your application.
17 //
18 // TRY THIS: Try running your application with "flutter run". You'll see
19 // the application has a purple toolbar. Then, without quitting the app,
20 // try changing the seedColor in the colorScheme below to Colors.green
21 // and then invoke "hot reload" (save your changes or press the "hot
22 // reload" button in a Flutter-supported IDE, or press "r" if you used
23 // the command line to start the app).
24 //
25 // Notice that the counter didn't reset back to zero; the application
26 // state is not lost during the reload. To reset the state, use hot
27 // restart instead.
28 //
29 // This works for code too, not just values: Most code changes can be
30 // tested with just a hot reload.
31 colorScheme: .fromSeed(seedColor: Colors.deepPurple),
32), // ThemeData
33 home: const MyHomePage(title: 'Flutter Demo Home Page'),
34); // MaterialApp
35 }
36 }
37
38 class MyHomePage extends StatefulWidget {
39 const MyHomePage({super.key, required this.title});
40
41 // This widget is the home page of your application. It is stateful, meaning
42 // that it has a State object (defined below) that contains fields that affect
43 // how it looks.
44
45 // This class is the configuration for the state. It holds the values (in this
46 // case the title) provided by the parent (in this case the App widget) and
47 // used by the build method of the State. Fields in a Widget subclass are
```

65) Ahora vamos a ejecutar la Aplicación realizado en Flutter con Dart. Para ello podemos Presiona F5 o ve a Ejecutar. O pulsa sobre **Run** como puedes ver en la siguiente imagen.



66) Ahora si queremos cambiar de dispositivo en el que queremos que se ejecute nuestra Aplicación Flutter. Tenemos dos opciones.

67) Con la paleta de comandos, para ello abrela presionando **Ctrl+Shift+P (Windows/Linux)** o **Cmd+Shift+P (macOS)**. Escribe Flutter: Select device y selecciona el comando. Se mostrará una lista de los dispositivos conectados y emuladores disponibles. Haz clic en el dispositivo que deseas usar.

68) Usando la barra de estado, para ello Busca la barra de estado en la parte inferior derecha de VS Code. Haz clic en el nombre del dispositivo que se muestra actualmente (por ejemplo, "No device", o el nombre de un emulador o teléfono conectado). Se abrirá un menú desplegable con la lista de todos los dispositivos disponibles. Selecciona el dispositivo deseado de la lista.

